

Database Systems I / Information Systems I

Functional Dependencies and Normalization — Complementary Material

Christian Rauch

(changed: 2026-05-09)

Dependencies Capture Data Semantics

A **dependency** describes a constraint that must hold for *every valid instance* of a relation – it reflects the meaning of the data, not just the current snapshot.

Intuition

If we know an artist's *id*, we can always determine their *name*. This is not a coincidence in today's data – it must hold for all future data too.

- ▶ Dependencies are part of the *schema*, not the data.
- ▶ They guide database design: poor design leads to **anomalies**.
- ▶ Normalisation uses dependencies to eliminate these anomalies.

Database Anomalies – Redundancy

Consider the unnormalised relation:

ArtifactInfo(artifactId, title, value, artistId, artistName, collectionTitle, collectionDesc)

artifactId	title	value	artistId	artistName	collectionTitle	collectionDesc
A1	Water Lilies	120 000	P7	Monet	Impressionism	Captivating masterpieces
A2	Haystacks	85 000	P7	Monet	Impressionism	Captivating masterpieces
A3	The Thinker	200 000	P9	Rodin	Sculpture	Inspiring sculptures

Redundancy

artistName and collectionDesc are stored once per artifact, not once per artist or collection.

Database Anomalies – Update & Deletion

Update Anomaly

Changing a single real-world fact requires updating *multiple* rows.

- ▶ If artist P7 changes their display name, every row with *artistId* = P7 must be updated.
- ▶ Missing one row leaves the database **inconsistent**.

Deletion Anomaly

Deleting a tuple may unintentionally destroy unrelated information.

- ▶ If A3 is the only artifact by artist P9, deleting it also destroys all knowledge of P9's name.
- ▶ Deleting the last artifact in a collection erases the collection's topic.

Root cause: Unrelated facts (artifact, artist, collection) are lumped into one relation.

Decomposition

Decomposition splits a relation into smaller, well-defined relations to eliminate anomalies.

ArtifactInfo can be decomposed into:

- ▶ *Artifacts*(artifactId, title, value, artistId, collectionTitle)
- ▶ *Artists*(artistId, artistName)
- ▶ *Collections*(collectionTitle, collectionDesc)

Goal

Each relation stores facts about exactly *one* concept, guided by the functional dependencies of the domain.

A good decomposition must be:

- ▶ **Lossless-join** – original data can be reconstructed exactly.
- ▶ **Dependency-preserving** – all constraints remain enforceable.
- ▶ **Redundancy-free** – no fact is stored more than once.

Functional Dependencies

Let R be a relation schema and $\alpha, \beta \subseteq R$.

Functional Dependency $\alpha \rightarrow \beta$

α **functionally determines** β if, in every valid instance r of R :

$$\forall t_1, t_2 \in r: \quad t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- ▶ An FD is a **schema property** – it must hold for every possible instance, not just the current one.
- ▶ It cannot be inferred from data alone; it must be known from the *data domain*.
- ▶ **Examples:**

$artistId \rightarrow artistName, \quad collectionTitle \rightarrow collectionDesc,$
 $artifactId \rightarrow title$

Keys and Superkeys

FDs generalise the notion of a key.

Superkey

A set $S \subseteq R$ is a **superkey** if $S \rightarrow R$, i.e. S functionally determines every attribute of R .

Key (Candidate Key)

A set $K \subseteq R$ is a **key** if it is a *minimal* superkey: $K \rightarrow R$ and no proper subset of K also determines R .

- ▶ Every key is a superkey; a superkey that is not minimal is *not* a key.
- ▶ **Example:** In *Artifacts*, $\{artifactId\}$ is a key; $\{artifactId, title\}$ could be a superkey but *not* a key (not minimal).

Trivial and Non-Trivial FDs

Trivial FD

$\alpha \rightarrow \beta$ is **trivial** if $\beta \subseteq \alpha$. It holds in every relation and carries no information.

Non-Trivial FD

$\alpha \rightarrow \beta$ is **non-trivial** if $\beta \not\subseteq \alpha$. It is **completely non-trivial** if $\alpha \cap \beta = \emptyset$.

- ▶ Trivial: $artistId, artistName \rightarrow artistId$
- ▶ Completely non-trivial: $artistId \rightarrow artistName$

Nonreducible FDs (Minimal Cover)

A set of FDs F_c is a **minimal cover** (canonical, nonreducible) of F if $F_c^+ = F^+$ and:

1. Every right-hand side contains exactly *one* attribute.
2. No FD in F_c is redundant (removing it changes F_c^+).
3. No left-hand side attribute is extraneous (removing it from any LHS does not change F_c^+).

Motivation

A minimal cover retains exactly the information needed – no more, no less. Schema-synthesis algorithms use it to produce a clean 3NF decomposition.

Note: A minimal cover need not be unique.

Armstrong's Axioms – Sound and Complete

Three inference rules form the foundation for deriving all FDs implied by a set F .

Reflexivity

If $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$. *(trivial FDs)*

Augmentation

If $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$ for any attribute set γ .

Transitivity

If $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$.

- ▶ **Sound:** every FD derivable by these rules genuinely holds.
- ▶ **Complete:** every FD that holds can be derived by these rules.

Derived Rules

The following rules are convenient shorthands; all are derivable from Armstrong's axioms.

Union

If $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$, then $\alpha \rightarrow \beta\gamma$.

Decomposition

If $\alpha \rightarrow \beta\gamma$, then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$.

Pseudotransitivity

If $\alpha \rightarrow \beta$ and $\gamma\beta \rightarrow \delta$, then $\alpha\gamma \rightarrow \delta$.

Closure of a Set of FDs

Closure F^+

The **closure** of F , written F^+ , is the set of *all* FDs that can be derived from F using Armstrong's axioms.

- ▶ F^+ may be exponentially larger than F .
- ▶ Two sets F and G are **equivalent** if $F^+ = G^+$.
- ▶ We rarely enumerate F^+ directly; instead we use the *attribute closure* to answer specific questions efficiently.

Attribute Closure α^+

Definition

The **attribute closure** of α under F , written α^+ , is the set of all attributes functionally determined by α :

$$\alpha^+ = \{ B \mid F \models \alpha \rightarrow B \}$$

Key uses of α^+ :

- ▶ $\alpha \rightarrow \beta$ holds under $F \iff \beta \subseteq \alpha^+$.
- ▶ α is a superkey $\iff \alpha^+ = R$.
- ▶ α is a key $\iff \alpha^+ = R$ and no proper subset of α has the same property.

Algorithm for Computing α^+

Attribute Closure Algorithm

Input: attribute set α , set of FDs F

$\alpha^+ := \alpha$

while α^+ *changes* **do**

for each $\beta \rightarrow \gamma$ **in** F **do**

if $\beta \subseteq \alpha^+$ **then** $\alpha^+ := \alpha^+ \cup \gamma$

Example:

$F = \{artifactId \rightarrow artistId\ title, artistId \rightarrow artistName\}$

Compute $\{artifactId\}^+$:

1. $\alpha^+ = \{artifactId\}$
2. Apply $artifactId \rightarrow artistId\ title$:
 $\alpha^+ = \{artifactId, artistId, title\}$
3. Apply $artistId \rightarrow artistName$:
 $\alpha^+ = \{artifactId, artistId, title, artistName\}$

Attribute Classification

Partition the attributes of R by how they appear in the FDs of F :

Three Disjoint Sets ($\mathcal{L} \cup \mathcal{B} \cup \mathcal{R} = R$)

- ▶ $\mathcal{L}$ – appears *only on the left* of some FD
- ▶ $\mathcal{R}$ – appears *only on the right* of some FD
- ▶ $\mathcal{B}$ – appears on *both* sides of some FDs

Observations:

- ▶ $\mathcal{L}$: must be in *every* key – no FD can produce an $\mathcal{L}$ -attribute, so it can only come from the key itself.
- ▶ $\mathcal{R}$: appears in *no* key – it is always derivable from others; including it would violate minimality.
- ▶ $\mathcal{B}$: *may or may not* appear in a given key.

Example:

$F = \{b \rightarrow a, c \rightarrow a, c \rightarrow f, f \rightarrow a, bc \rightarrow d, bc \rightarrow e, de \rightarrow b\}$

$\mathcal{L} = \{c\}, \quad \mathcal{R} = \{a\}, \quad \mathcal{B} = \{b, d, e, f\}$

Attribute Classification – Museum Example

Exhibitions(*eid*, *date*, *cid*, *cname*, *room*) (*eid* = exhibitionId,
cid = curatorId, *cname* = curatorName)

FDs: $eid \rightarrow date, cid, room, \quad cid \rightarrow cname, \quad cid, date \rightarrow eid$

Classify each attribute:

Attribute	Sides	Set
<i>eid</i>	left ($eid \rightarrow \dots$) and right ($cid, date \rightarrow eid$)	$\mathcal{B}$
<i>date</i>	left ($cid, date \rightarrow eid$) and right ($eid \rightarrow date$)	$\mathcal{B}$
<i>cid</i>	left ($cid \rightarrow cname, \dots$) and right ($eid \rightarrow cid$)	$\mathcal{B}$
<i>cname</i>	right only ($cid \rightarrow cname$)	$\mathcal{R}$
<i>room</i>	right only ($eid \rightarrow room$)	$\mathcal{R}$

$\mathcal{L} = \emptyset, \quad \mathcal{R} = \{cname, room\}, \quad \mathcal{B} = \{eid, date, cid\}$

- ▶ $\mathcal{L} = \emptyset$: no attribute is forced into every key; all candidate key attributes come from $\mathcal{B}$.
- ▶ *cname* and *room* will never appear in any key.

Algorithm for Computing All Keys

FINDALLKEYS(R, F) (C is a set of candidate attribute sets; $\gamma \subseteq R$)

(1) Compute $\mathcal{L}, \mathcal{R}, \mathcal{B}$.

(2) Init: $C \leftarrow \{\mathcal{L}\}, \text{Keys} \leftarrow \emptyset$.

(C starts with one candidate: $\mathcal{L}$ itself; if $\mathcal{L} = \emptyset$ then $C = \{\emptyset\}$, i.e. one candidate, the empty set)

(3) **while** $C \neq \emptyset$ **do**

for each set $\gamma \in C$: **if** $\gamma^+ = R$ **then** move γ to Keys .

 Expand: pair every remaining $\gamma \in C$ with every attribute $x \in \mathcal{B} \setminus \gamma$;

 discard any result that is a superset of a known key;

 replace C with the surviving candidates:

$C \leftarrow \{\gamma \cup \{x\} \mid \gamma \in C, x \in \mathcal{B} \setminus \gamma, \nexists k \in \text{Keys} : k \subseteq \gamma \cup \{x\}\}$

(4) **return** Keys

Each Expand replaces C with candidates that are *one attribute larger*. The while loop therefore performs a *Breadth-First Search* over candidate sets ordered by size: iteration 1 tests all size- $|\mathcal{L}| + 1$ sets, iteration 2 all size- $|\mathcal{L}| + 2$ sets, and so on. Once every candidate is either a key or a superset of a known key, C becomes empty and the loop stops.

Finding All Keys – Museum Example

Same *Exhibitions* relation; $\mathcal{L} = \emptyset$, $\mathcal{B} = \{eid, date, cid\}$.

- ▶ **Init:** $C = \{\emptyset\}$; $\emptyset^+ = \emptyset \neq R$
- ▶ **Expand** from $\emptyset$: $C = \{\{eid\}, \{date\}, \{cid\}\}$
- ▶ **Check closures:**
 - ▶ $\{eid\}^+ = \{eid, date, cid, room, cname\} = R \Rightarrow \text{key}$
 - ▶ $\{date\}^+ = \{date\} \neq R$
 - ▶ $\{cid\}^+ = \{cid, cname\} \neq R$
- ▶ **Expand** $\{date\}$ and $\{cid\}$ (skip adding eid : would create a superset of key $\{eid\}$): $C = \{\{cid, date\}\}$
- ▶ **Check:** $\{cid, date\}^+$: apply $cid \rightarrow cname \rightarrow$ apply $cid, date \rightarrow eid \rightarrow$ apply $eid \rightarrow date, cid, room$: $= R \Rightarrow \text{key}$
- ▶ **It. 3:** all further expansions are supersets of a key $\rightarrow C = \emptyset$

Keys = $\{\{eid\}, \{cid, date\}\}$

Properties of a Good Decomposition

A decomposition of R into $\{R_1, R_2, \dots, R_n\}$ should satisfy:

1 – Lossless-Join Decomposition

The original relation can be recovered exactly by natural join:

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_n}(r)$$

No spurious tuples are introduced.

2 – Dependency Preservation

Every FD in F^+ can be checked using only the individual decomposed relations, without performing joins.

3 – No Redundancy

Each relation stores facts about a single concept; no information is unnecessarily repeated across tuples.

Lossless-Join Decomposition

Test for Binary Decomposition

A decomposition of R into R_1 and R_2 is **lossless-join** if and only if at least one holds:

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{or} \quad (R_1 \cap R_2) \rightarrow R_2$$

- ▶ The shared attributes must be a superkey of at least one part.
- ▶ Joining $\pi_{R_1}(r) \bowtie \pi_{R_2}(r)$ then yields exactly r – no spurious tuples.

Example: Decompose *ArtifactInfo* into *Artifacts* and *Artists*.

$R_1 \cap R_2 = \{artistId\}$, and $artistId \rightarrow artistName$ (i.e.

$R_1 \cap R_2 \rightarrow R_2$). *Lossless.*

Dependency Preservation

Definition

Let F_i be the projection of F^+ onto R_i . The decomposition is **dependency-preserving** if:

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

- ▶ If dependency preservation fails, some FDs can only be checked by performing expensive joins.
- ▶ Not all normal-form decompositions are dependency-preserving (e.g. BCNF may sacrifice this property).
- ▶ **3NF** always admits a lossless *and* dependency-preserving decomposition.

Multivalued Dependencies

Let R be a relation schema and $\alpha, \beta \subseteq R$.

Multivalued Dependency $\alpha \twoheadrightarrow \beta$

α **multidetermines** β if, for every valid instance r of R and every pair $t_1, t_2 \in r$ with $t_1[\alpha] = t_2[\alpha]$, there exists $t_3 \in r$ such that:

$$t_3[\alpha] = t_1[\alpha], \quad t_3[\beta] = t_1[\beta], \quad t_3[R \setminus \alpha\beta] = t_2[R \setminus \alpha\beta]$$

- ▶ The set of β -values associated with an α -value is *independent* of the remaining attributes.
- ▶ Every FD $\alpha \rightarrow \beta$ implies $\alpha \twoheadrightarrow \beta$.

Multivalued Dependencies – Properties

Complementation

If $\alpha \twoheadrightarrow \beta$, then $\alpha \twoheadrightarrow R \setminus \alpha\beta$.

Trivial MVD

$\alpha \twoheadrightarrow \beta$ is **trivial** if $\beta \subseteq \alpha$ or $\alpha \cup \beta = R$.

- ▶ Non-trivial MVDs cause redundancy that FDs alone cannot capture.
- ▶ They motivate **Fourth Normal Form (4NF)**: a relation is in 4NF if, for every non-trivial $\alpha \twoheadrightarrow \beta$, α is a superkey.
- ▶ **Example:** *artifactId* $\twoheadrightarrow$ *authorId* and *artifactId* $\twoheadrightarrow$ *exhibitionTitle* – independently associated sets.

Normalization – Overview

Goal

Normalization is the process of decomposing relation schemas to eliminate redundancy and anomalies caused by undesirable functional (and multivalued) dependencies.

- ▶ A series of **normal forms** (1NF, 2NF, 3NF, BCNF, 4NF, ...) defines increasingly strict requirements.
- ▶ Each higher normal form is a *subset* of the previous:
 $4NF \subset BCNF \subset 3NF \subset \dots$
- ▶ Normalization guides us toward a *canonical design* – a relation per concept, attributes depending *only* on the key.
- ▶ A good decomposition must be **lossless** and, ideally, **dependency-preserving**.

Boyce-Codd Normal Form (BCNF)

Definition

A relation schema R is in **BCNF** with respect to F if, for every non-trivial FD $\alpha \rightarrow \beta$ in F^+ :

α is a superkey of R .

- ▶ Intuitively: every non-trivial FD must be *determined by a superkey* – there are no “hidden” dependencies on partial or unrelated attribute sets.
- ▶ BCNF **eliminates all redundancy** due to functional dependencies.
- ▶ A lossless BCNF decomposition always exists; it may, however, *not* be dependency-preserving.

BCNF – Decomposition Algorithm

Algorithm

while $\exists$ a non-trivial, non-reducible $\alpha \rightarrow \beta$ in F^+ where α is *not* a superkey of the current schema R **do**

 Collect all FDs in F^+ with the same LHS α :

$\alpha \rightarrow \beta_1, \dots, \alpha \rightarrow \beta_k$

 Decompose R into:

$R_1 = \pi_{\alpha, \beta_1, \dots, \beta_k}(R)$ (lossless: α is a key of R_1)

$R_2 = \pi_{R \setminus \{\beta_1, \dots, \beta_k\}}(R)$

- ▶ Termination is guaranteed (finitely many LHS sets).
- ▶ The algorithm requires the attribute closure α^+ to identify all violated FDs.

BCNF – Example

ExamSlot(artifactId, date, timeSlot, curatorId, room)

Keys: {*artifactId*, *date*}, {*curatorId*, *date*, *timeSlot*},
{*room*, *date*, *timeSlot*}

Non-reducible FDs:

- ▶ *artifactId*, *date* → *timeSlot*, *curatorId*, *room*
- ▶ *curatorId*, *date*, *timeSlot* → *artifactId*
- ▶ *room*, *date*, *timeSlot* → *artifactId*
- ▶ *curatorId*, *date* → *room* (non-trivial; LHS not a superkey!)

Decompose on *curatorId*, *date* → *room*:

- ▶ $R_1(\textit{curatorId}, \textit{date}, \textit{room})$ – lossless, {*curatorId*, *date*} is key of R_1
- ▶ $R_2(\textit{artifactId}, \textit{date}, \textit{timeSlot}, \textit{curatorId})$ – check for further violations

BCNF – Dependency Preservation Trade-off

BCNF decomposition is **not always dependency-preserving**.

Example: *ExhibitionVenue*(*venue*, *city*, *artistId*)

- ▶ FDs: $venue \rightarrow city$, $artistId, city \rightarrow venue$
- ▶ Keys: $\{artistId, city\}$, $\{artistId, venue\}$
- ▶ Not in BCNF: $venue \rightarrow city$ has a non-superkey LHS.
- ▶ Any decomposition *loses* $artistId, city \rightarrow venue$; enforcing it would require a join.

Consequence

When dependency preservation is required, use **3NF** instead – it always admits a lossless *and* dependency-preserving decomposition.

Third Normal Form (3NF)

Definition

A relation schema R is in **3NF** with respect to F if, for every non-trivial FD $\alpha \rightarrow B$ in F^+ , at least one holds:

1. α is a superkey of R , **or**
 2. B is contained in some key of R (B is a *prime attribute*).
- ▶ Every BCNF relation is also in 3NF (BCNF is stricter).
 - ▶ 3NF allows limited redundancy – specifically when a non-superkey determines a prime attribute.
 - ▶ **3NF always admits a lossless and dependency-preserving decomposition.**

3NF – Redundancy Trade-off

Example: *ExhibitionVenue*(venue, city, artistId)

- ▶ FDs: $venue \rightarrow city$, $artistId, city \rightarrow venue$
- ▶ Keys: $\{artistId, city\}$, $\{artistId, venue\}$

ExhibitionVenue **is in 3NF:**

- ▶ $venue \rightarrow city$: city is a prime attribute (part of a key). ✓
- ▶ $artistId, city \rightarrow venue$: LHS is a key, hence a superkey. ✓

venue	city	artistId
Grand Palais	Paris	Monet
Louvre	Paris	Rodin
Grand Palais	Paris	Renoir

$venue \rightarrow city$ forces Paris to repeat – *some redundancy remains.*

3NF Synthesis Algorithm

Produces a lossless, dependency-preserving 3NF decomposition.

Algorithm

Let F_c be a minimal cover of F .

- (1) For each $\alpha \rightarrow a \in F_c$, create $R_i = \alpha \cup \{a\}$.
- (2) If no R_i contains a superkey of R , add a subrelation whose schema is any key of R .
- (3) Combine subrelations with the same LHS α .
- (4) Drop any R_i that is a proper subset of some R_j .

Example: $F_c = \{artistId \rightarrow artistName, collectionTitle \rightarrow collectionDesc, artifactId \rightarrow artistId\}$
Step (1) yields *Artists*, *Collections*, *Artifacts* – $\{artifactId\}$ is already a key of R , so step (2) is skipped.

4NF – Fourth Normal Form

Definition

A relation schema R is in **4NF** if, for every non-trivial MVD $\alpha \twoheadrightarrow \beta$ in F^+ , α is a superkey of R .

Example: *ArtistWork(artistId, exhibitionTitle, artifactId)*

- ▶ No non-trivial FDs $\Rightarrow$ relation is in BCNF.
- ▶ But: $artistId \twoheadrightarrow exhibitionTitle$ and $artistId \twoheadrightarrow artifactId$ (independently associated sets).
- ▶ Adding a new exhibition for an artist requires inserting one row per artifact – **insertion anomaly**.
- ▶ Decompose into $(artistId, exhibitionTitle)$ and $(artistId, artifactId)$, both in 4NF.

Normal Forms – Comparison

Property	3NF	BCNF	4NF
Removes redundancy due to FDs	no	yes	yes
Removes redundancy due to MVDs	no	no	yes
Lossless decomposition	yes	yes	yes
Dependency preservation	yes	no	no

- ▶ $4NF \subset BCNF \subset 3NF$: every 4NF relation is in BCNF, every BCNF relation is in 3NF – but not vice versa.
- ▶ **BCNF** is the standard target in practice.
- ▶ **3NF** is preferred when dependency preservation is required and some redundancy is acceptable.
- ▶ **4NF** further eliminates MVD-induced redundancy.